



ESTRUTURA DE DADOS

LISTAS ENCADEADAS

Prof. Me. Gustavo Ferreira Palma

PAUTA

- Listas Encadeadas

LISTAS ENCADEADAS

LISTAS ENCADEADAS - CONCEITO

- Vetores possuem tamanho fixo e armazenam seus elementos em posições contíguas da memória. Embora sejam simples e eficientes para acesso direto aos elementos, apresentam limitações quando é necessário inserir ou remover dados durante a execução do programa.
- Além disso, inserir ou remover elementos em um vetor normalmente exige deslocar diversos elementos, tornando essas operações menos eficientes.
- Uma lista encadeada é uma estrutura dinâmica composta por elementos chamados nós. Cada nó armazena um dado e o endereço do próximo elemento da lista.

LISTAS ENCADEADAS - CONCEITO

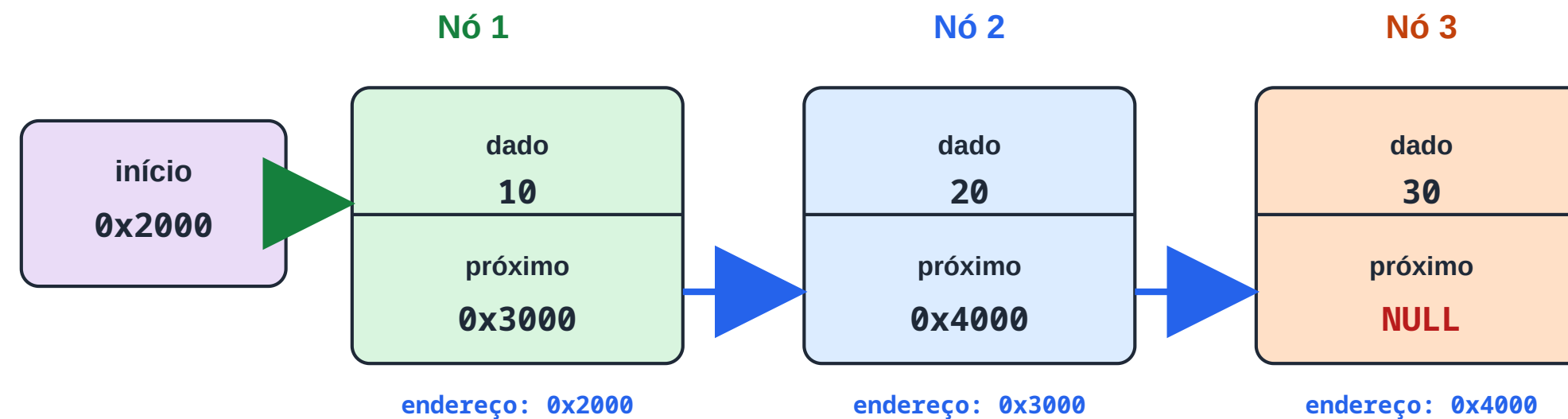
- Estrutura de um Nó

```
1 //...
2 typedef struct No
3 {
4     int dado; // informação armazenada
5     struct No *proximo;
6 } No;
7 //...
```


LISTAS ENCADEADAS - CONCEITO

Lista encadeada simples

Cada nó armazena um dado e o endereço do próximo nó da lista.



início armazena o endereço do primeiro nó

NULL indica o fim da lista

O campo próximo liga um nó ao seguinte

```
typedef struct No { int dado; struct No *proximo; } No;
```

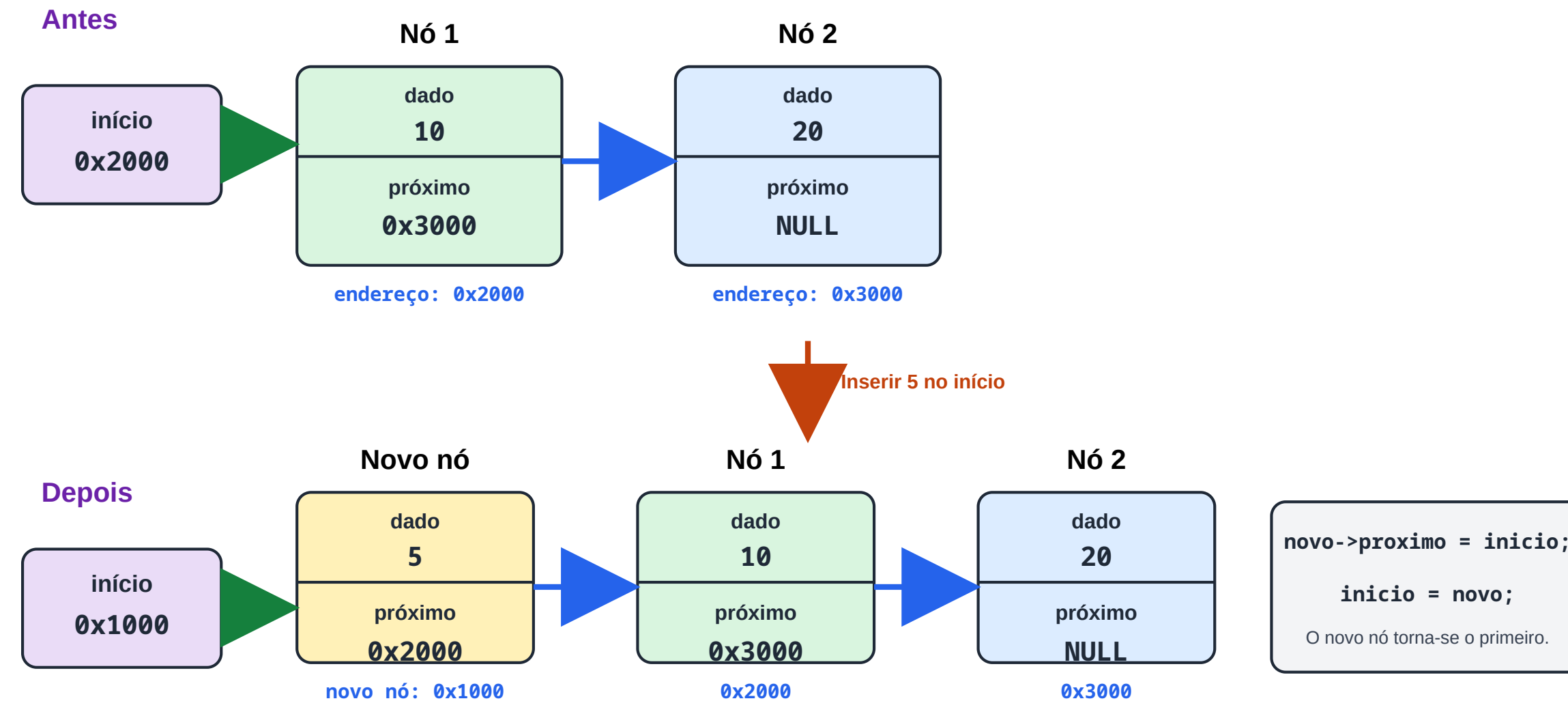
Os nós não precisam estar contíguos na memória; os ponteiros mantêm a sequência lógica.

LISTAS ENCADEADAS - UTILIZAÇÃO

- Inserção de dados no início da lista

Inserção no início da lista

O novo nó aponta para o antigo primeiro nó, e início passa a apontar para ele.



LISTAS ENCADEADAS - CONCEITO

- Inserção de dados no início da lista

```
1 //...
2 No *novo = malloc(sizeof(No));
3
4 novo->proximo = inicio;
5
6 inicio = novo;
7 //...
```

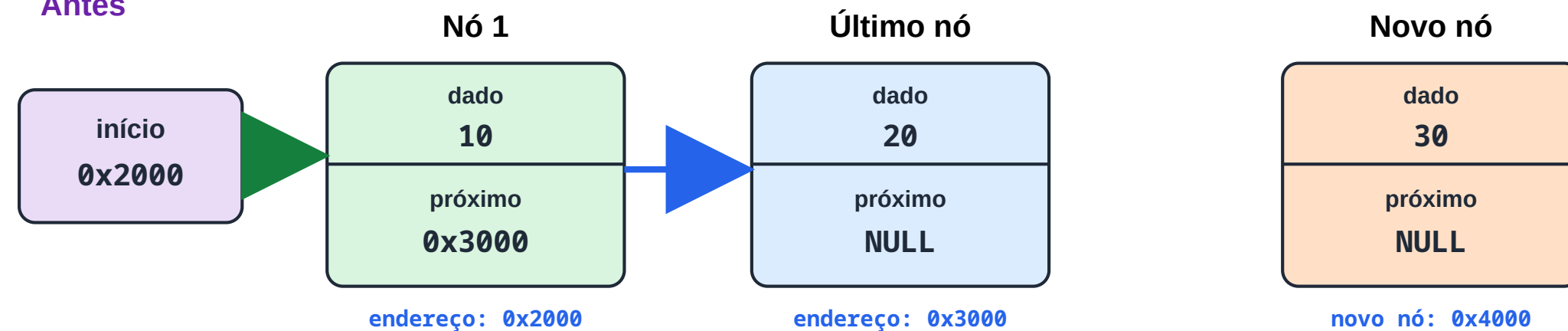

LISTAS ENCADEADAS - UTILIZAÇÃO

- Inserção de dados no final da lista

Inserção no final da lista

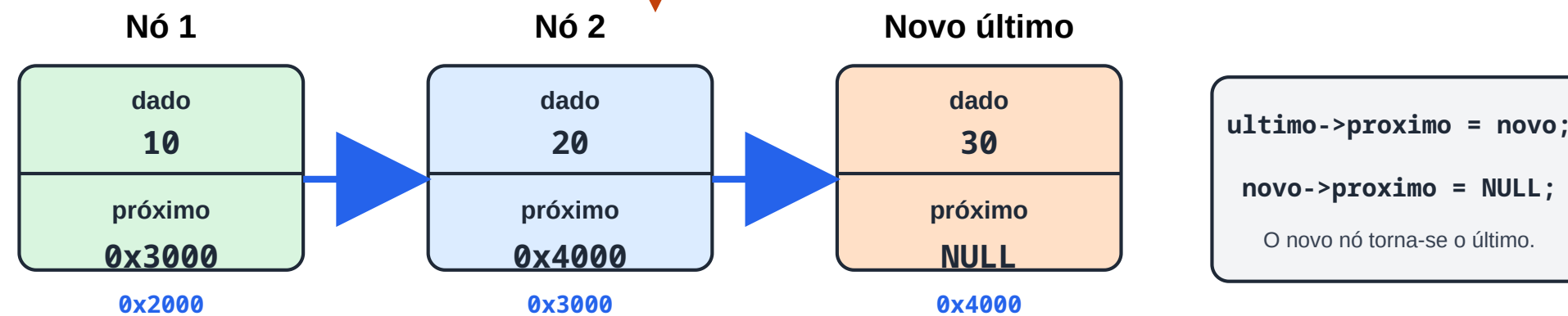
Localize o último nó e faça seu campo próximo apontar para o novo nó.

Antes



Percorrer até o último nó

Depois



LISTAS ENCADEADAS - UTILIZAÇÃO

- Inserção de dados no final da lista

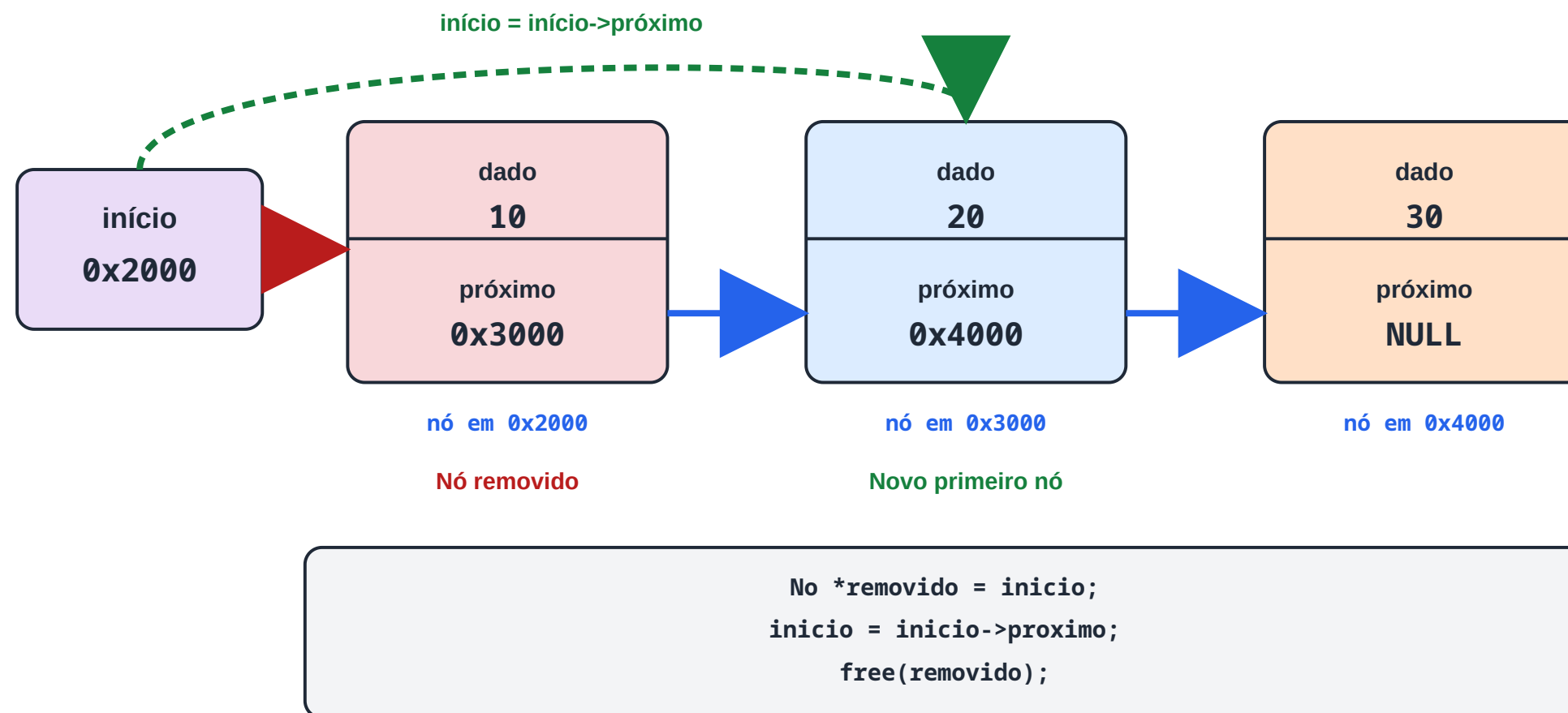
```
1 //...
2 while(atual->proximo != NULL)
3 {
4     atual = atual->proximo;
5 }
6
7 atual->proximo = novo;
8 //...
```

LISTAS ENCADEADAS - UTILIZAÇÃO

- Remoção de dados no início da lista

Remoção no início da lista

O ponteiro início passa a apontar para o segundo nó e o primeiro nó é liberado.



LISTAS ENCADEADAS - UTILIZAÇÃO

- Remoção de dados no início da lista

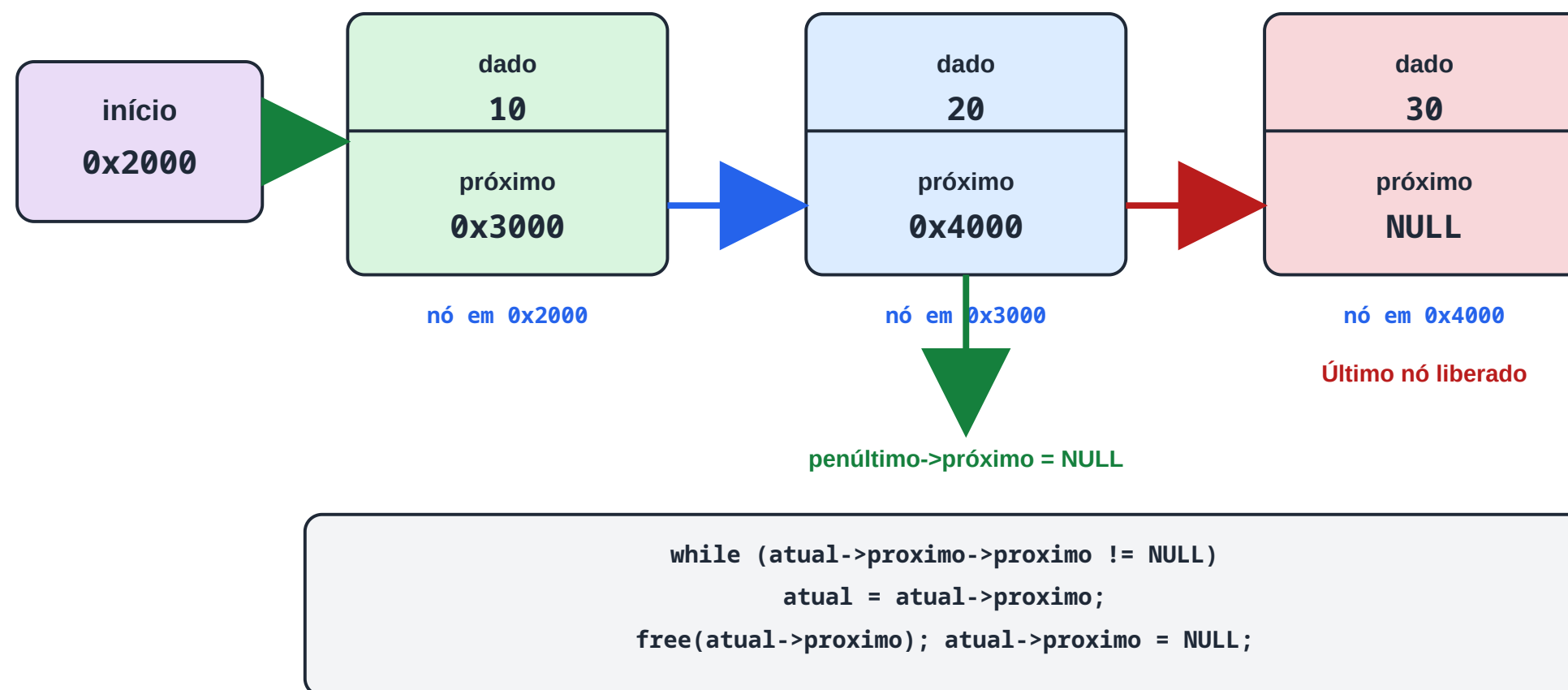
```
1 //...
2     if (*inicio == NULL)
3     {
4         return;
5     }
6
7     No *temp = *inicio;
8
9     *inicio = (*inicio)->proximo;
10
11     free(temp);
12 //...
```

LISTAS ENCADEADAS - UTILIZAÇÃO

- Remoção de dados no final da lista

Remoção no final da lista

É necessário percorrer a lista até o penúltimo nó.



LISTAS ENCADEADAS - UTILIZAÇÃO

- Remoção de dados no final da lista

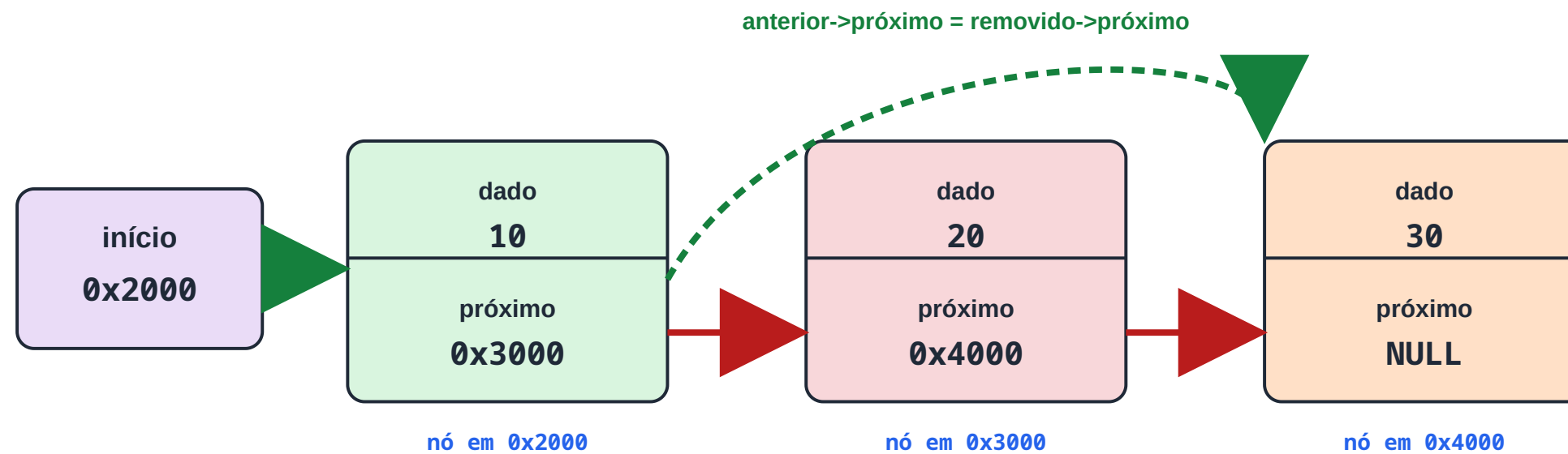
```
1 //...
2 if (*inicio == NULL)
3 {
4     return;
5 }
6
7 if ((*inicio)->proximo == NULL)
8 {
9     free(*inicio);
10    *inicio = NULL;
11    return;
12 }
13
14 No *anterior = NULL;
15 No *atual = *inicio;
```

LISTAS ENCADEADAS - UTILIZAÇÃO

- Remoção de dados em posições da lista

Remoção em uma posição da lista

Exemplo: remover o nó da posição 1, contendo o valor 20.



Nó removido da posição 1

```
No *removido = anterior->proximo;  
anterior->proximo = removido->proximo;  
free(removido);
```

É necessário localizar o nó anterior à posição que será removida.

LISTAS ENCADEADAS - UTILIZAÇÃO

- Remoção de dados em posições da lista

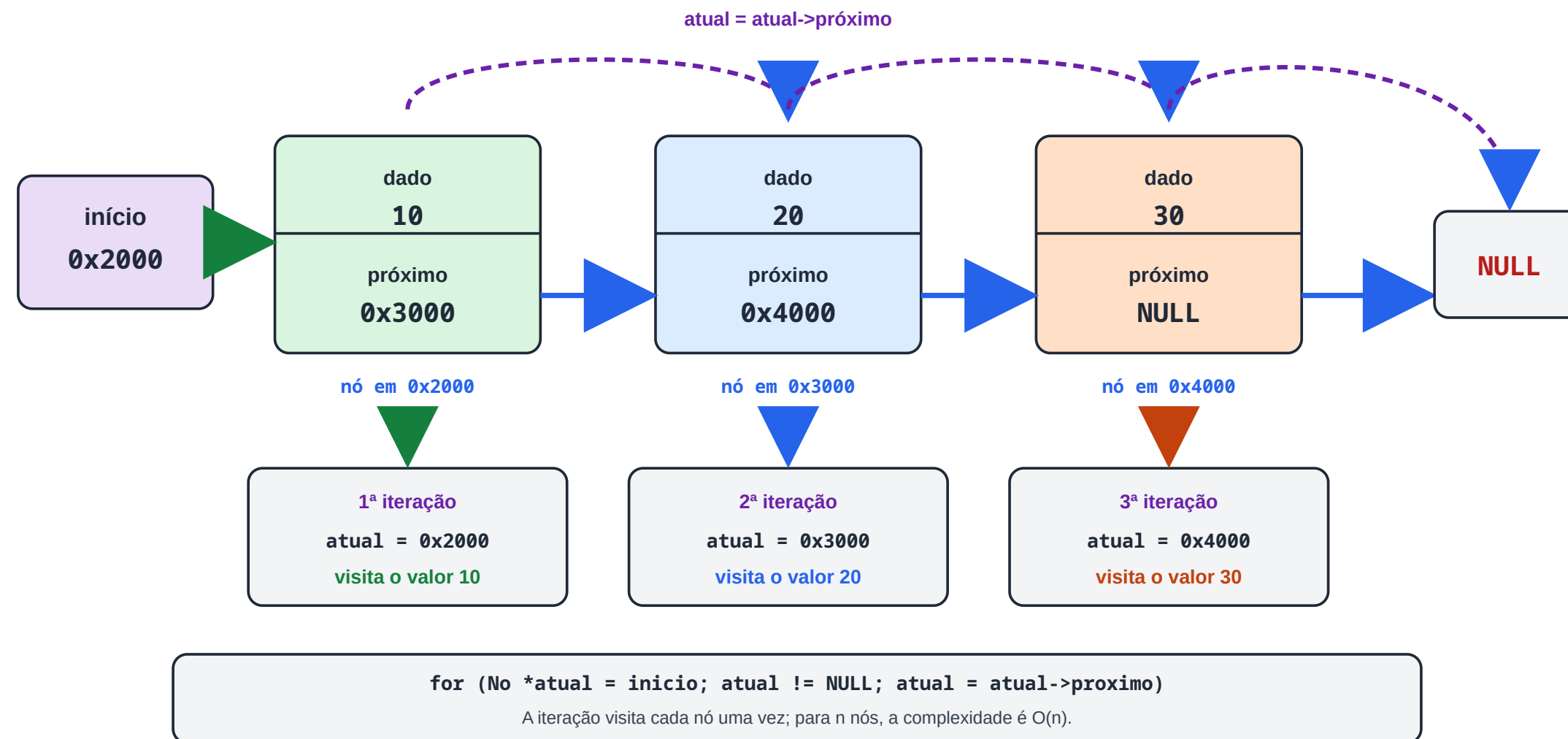
```
1 //...
2 if (*inicio == NULL)
3 {
4     return;
5 }
6
7 if (posicao == 0)
8 {
9     removerInicio(inicio);
10    return;
11 }
12
13 No *anterior = NULL;
14 No *atual = *inicio;
15
```


LISTAS ENCADEADAS - UTILIZAÇÃO

- Iterando uma Lista

Iteração em uma lista encadeada

O ponteiro atual começa no início e avança pelos campos próximo até alcançar NULL.



LISTAS ENCADEADAS - UTILIZAÇÃO

- Iterando uma Lista

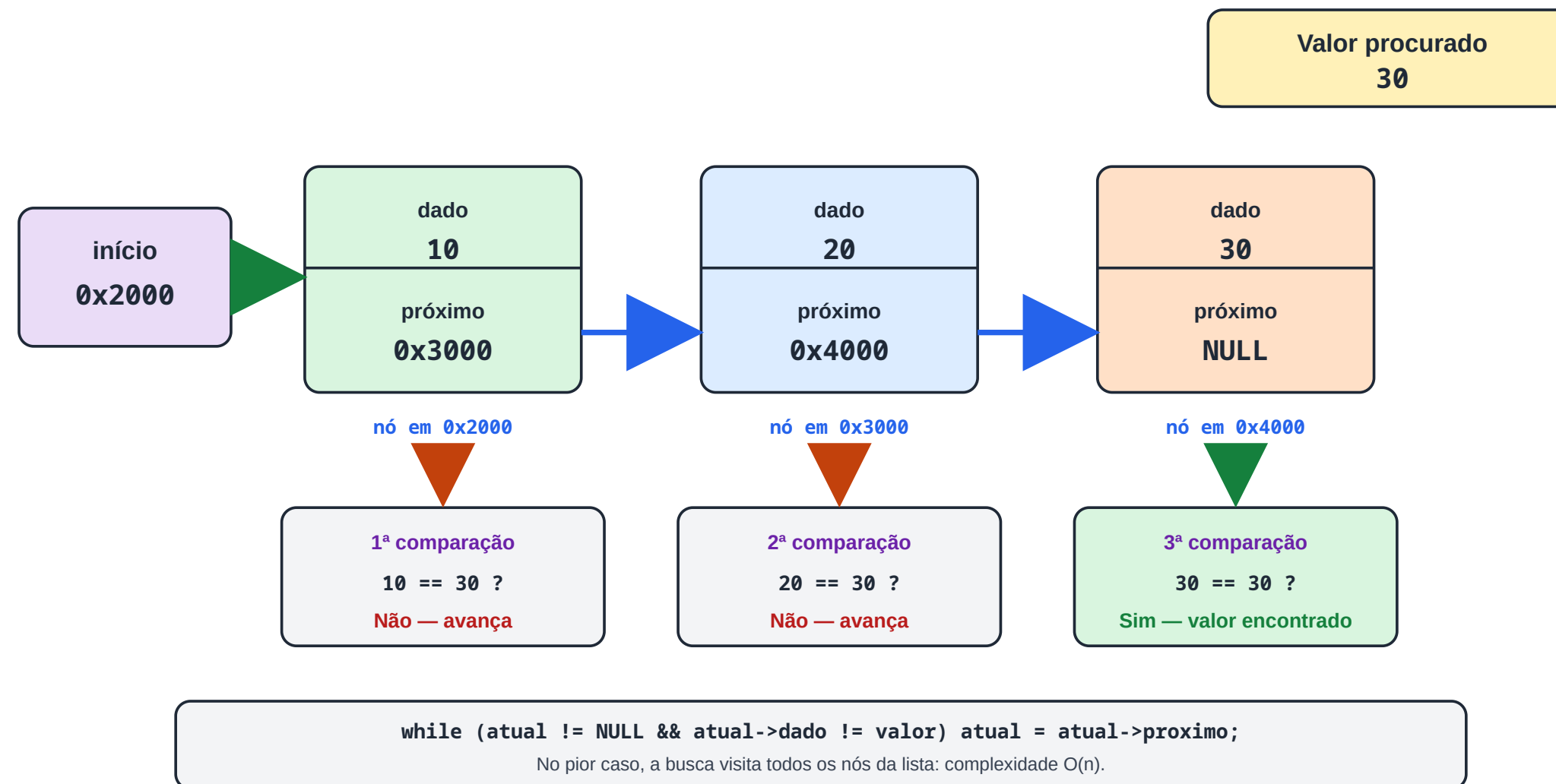
```
1 //...
2 No *atual = inicio;
3
4 while(atual != NULL)
5 {
6     printf("%d", atual->dado);
7
8     atual = atual->proximo;
9 }
10 //...
```

LISTAS ENCADEADAS - UTILIZAÇÃO

- Buscando valores em listas

Busca sequencial em uma lista encadeada

Exemplo: procurar o valor 30, visitando os nós a partir do início.



LISTAS ENCADEADAS - UTILIZAÇÃO

- Buscando valores em listas

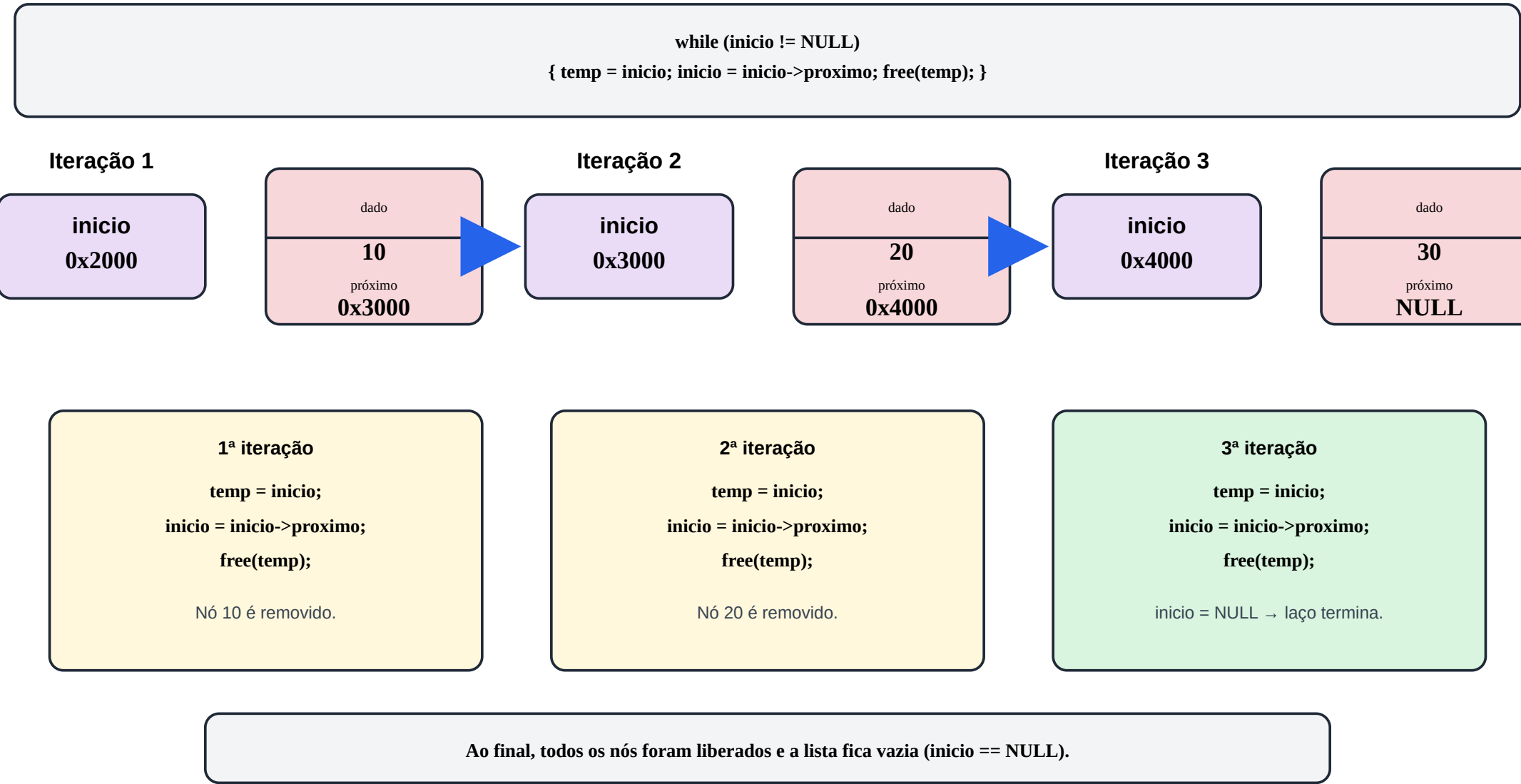
```
1 //...
2 int dadoBusca = 0;
3 printf("Insira o valor a ser buscado:\n");
4 scanf("%d", &dadoBusca);
5
6 while(atual != NULL)
7 {
8     if (atual->dado == dadoBusca ){
9         printf("Encontrado!\n");
10    }
11    atual = atual->proximo;
12 }
13 //...
```

LISTAS ENCADEADAS - UTILIZAÇÃO

- Liberando a memória Alocada

Liberação completa de uma lista encadeada

Cada iteração remove apenas o primeiro nó da lista.



LISTAS ENCADEADAS - UTILIZAÇÃO

- Buscando valores em listas

```
1 //...
2 while(inicio != NULL)
3 {
4     No *temp = inicio;
5
6     inicio = inicio->proximo;
7
8     free(temp);
9 }
10 //...
```

OBRIGADO!

Podem me encontrar aqui:

- gustavo.palma@unisal.edu.br
- www.gustavopalma.com.br